CV programes

Ex 1:

Basic image processing, loading, cropping, resizing, threading, contour analysis , blob detection

```
import cv2 as cv
import numpy as np
img_path=r'AVATAR.jpg'
image=cv.imread(img_path)
cv.imshow('Original Image ',image)
cropped_img=image[50:300,100:400]
cv.imshow('original Image',image)

resized_img=cv.resize(image,(400,400))
cv.imshow('resized image',resized_img)

grayimg=cv.cvtColor(image,cv.COLOR_BGR2GRAY)
_,thresh_img=cv.threshold(grayimg,127,255,cv.THRESH_BINARY)
cv.imshow('threholded Image',thresh_img)

contours,_=cv.findContours(thresh_img,cv.RETR_TREE,cv.CHAIN_APPROX_SIMPLE)
contour_img=image.copy()
cv.drawContours(contour_img,contours,-1,(0,255,0),2)
```

```
cv.imshow('contour Analisis',contour_img)
params=cv.SimpleBlobDetector_Params()
params.filterByArea=True
params.minArea=30
params.maxArea=5000
params.filterByCircularity=False
params.filterByConvexity=False
params.filterByInertia=False
params.minThreshold=10
params.maxThreshold=200
detector=cv.SimpleBlobDetector_create(params)
keypoints=detector.detect(thresh_img)
blob_img=cv.drawKeypoints(image,keypoints,np.array([]),(0,0,255),
                    cv.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv.imshow('BLob Detection',blob_img)
cv.waitKey(0)
cv.destreoyALLWindows()
```

Exp 2:

Image annotation-Drawing line , circle, rectangle , ellips ,text on image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img=cv2.imread("GOKU.jpg")


img_rgb=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)

imageline=img_rgb.copy()
cv2.line(imageline,(75,100),(1500,1500),(0,255,0),30)

imagecircle=img_rgb.copy()
cv2.circle(imagecircle,(550,700),500,(100,0,0),15)

imagerect=img_rgb.copy()
cv2.rectangle(imagerect,(550,700),(10,500),(0,200,0),20)

imageellip=img_rgb.copy()
cv2.ellipse(imageellip,(550,700),(500,100),0,0,360,(200,0,0),20)
```

```
imagetext=img_rgb.copy()
cv2.putText(imagetext,'SON
GOKU',(100,1800),cv2.FONT_HERSHEY_SIMPLEX,5,(0,255,0),10,cv2.LINE_AA)

titles=['Original Image','LINE','CIRCLE','RECTANGLE','ELLIPSE','TEXT']

images=[img_rgb,imageline,imagecircle,imagerect,imageellip,imagetext]

plt.figure(figsize=(15,10))
for i in range(6):
    plt.subplot(2,3,i+1)
    plt.imshow(images[i])
    plt.title(titles[i])
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Exp 3:

Imsge enhancement -understanding color space conversion , histogram equalization , convolution , image smoothing , gradient and edge detection

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image=cv2.imread('G:\CV_008\goku1.jpg')
gray=cv2.cvtColor(image,cv2.COLOR_BGR2GRAY)
rgb=cv2.cvtColor(image,cv2.COLOR_BGR2RGB)
hsv=cv2.cvtColor(image,cv2.COLOR_BGR2HSV)
cmy=255-image
hist_eq=cv2.equalizeHist(gray)
blur_avg=cv2.blur(image,(5,5))
blur_gaussian=cv2.GaussianBlur(image,(5,5),0)
blur_median=cv2.medianBlur(image,5)
sharpen_kernel=np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharpened=cv2.filter2D(image,-1,sharpen_kernel)
sobelx=cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=3)
sobely=cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=3)
gradient_magnitude=cv2.magnitude(sobelx,sobely)
edges = cv2.Canny(gray,100,200)
plt.figure(figsize=(14,10))
```

```
plt.subplot(3,5,1)
plt.imshow(cv2.cvtColor(image,cv2.COLOR_BGR2RGB))
plt.title('Original')
plt.axis('off')
plt.subplot(3,5,2)
plt.imshow(cv2.cvtColor(rgb,cv2.COLOR_BGR2RGB))
plt.title('RGB')
plt.axis('off')
plt.subplot(3,5,3)
plt.imshow(cv2.cvtColor(hsv,cv2.COLOR_BGR2RGB))
plt.title('HSV')
plt.axis('off')
plt.subplot(3,5,4)
plt.imshow(cv2.cvtColor(cmy,cv2.COLOR_BGR2RGB))
plt.title('CMY')
plt.axis('off')
plt.subplot(3,5,5)
plt.imshow(hist_eq,cmap='gray')
plt.title('Histogram Equalization')
plt.axis('off')
plt.subplot(3,5,6)
plt.imshow(cv2.cvtColor(blur_avg,cv2.COLOR_BGR2RGB))
plt.title('Avg Blur')
plt.axis('off')
plt.subplot(3,5,7)
plt.imshow(cv2.cvtColor(blur_gaussian,cv2.COLOR_BGR2RGB))
```

```
plt.title('Gaussian Blur')
plt.axis('off')
plt.subplot(3,5,8)
plt.imshow(cv2.cvtColor(blur_median,cv2.COLOR_BGR2RGB))
plt.title('Median Blur')
plt.axis('off')
plt.subplot(3,5,9)
plt.imshow(cv2.cvtColor(sharpened,cv2.COLOR_BGR2RGB))
plt.title('Sharpened (Convolution)')
plt.axis('off')
plt.subplot(3,5,10)
plt.imshow(sobelx,cmap='gray')
plt.title('Sobel X')
plt.axis('off')
plt.subplot(3,5,11)
plt.imshow(sobely,cmap='gray')
plt.title('Sobel Y')
plt.axis('off')
plt.subplot(3,5,12)
plt.imshow(gradient_magnitude,cmap='gray')
plt.title('Gradient Magnitude')
plt.axis('off')
plt.subplot(3,5,13)
plt.imshow(edges,cmap='gray')
plt.title('Canny Edges')
plt.axis('off')
```

```
plt.subplot(3,5,14)
plt.hist(gray.ravel(), bins=256)
plt.title("Grayscale Image Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.subplot(3,5,15)
plt.hist(hist_eq.ravel(), bins=256)
plt.title("Grayscale Image Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

Exp 4:

Image features and image alignment image transform-Fourier,hough,extract ORBimage features, feature matching , cloning

Feature based image aligjnment

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# -------------------- Read Images --------------------
img = cv2.imread(r"G:\CV_008\Gohan.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

gray_img=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
rows, cols = gray_img.shape

clon=np.copy(img)



# -------------------- FFT --------------------
nrows = cv2.getOptimalDFTSize(rows)
ncols = cv2.getOptimalDFTSize(cols)

nimg = np.zeros((nrows, ncols))
```

```
nimg[:rows, :cols] = gray_img


mask = np.zeros((nrows, ncols), np.uint8)
x, y = np.meshgrid(np.arange(ncols), np.arange(nrows))
mask_area = ((x - ncols//2)**2 + (y - nrows//2)**2) < (nrows//4)**2
mask[mask_area] = 1


f = np.fft.fft2(nimg)
fshift = np.fft.fftshift(f)
fshift_mask = fshift * mask
magnitude_spectrum = 20 * np.log(np.abs(fshift_mask) + 1)


# -------------------- Hough Line Detection --------------------
edges = cv2.Canny(gray_img, 50, 150)
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 50, 50, 10)


line_img = img.copy()
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(line_img, (x1, y1), (x2, y2), (255, 0, 0), 9)


# -------------------- Hough Circle Detection --------------------
blurred = cv2.GaussianBlur(gray_img, (9, 9), 2)
circles = cv2.HoughCircles(
    blurred,
```

```
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=50,
    param2=30,
    minRadius=10,
    maxRadius=100
)

circle_img = img.copy()
if circles is not None:
    circles = np.uint16(np.around(circles))
    for x, y, r in circles[0]:
        cv2.circle(circle_img, (x, y), r, (0, 255, 0), 2)
        cv2.circle(circle_img, (x, y), 2, (255, 0, 0), 3)

# -------------------- ORB Keypoints --------------------
orb = cv2.ORB_create(nfeatures=1500)
keypoints, descriptors = orb.detectAndCompute(img, None)
img_keypoints = cv2.drawKeypoints(img, keypoints, None)
img_keypoints=img.copy()
for kp in keypoints:
    x,y=int(kp.pt[0]),int(kp.pt[1])
    cv2.circle(img_keypoints,(x,y),radius=1,color=(255,0,0),thickness=9)

# -------------------- Feature Matching --------------------
```

```
img1 = cv2.imread(r"G:\CV_008\Gohan.jpg")
img2 = cv2.imread(r"G:\CV_008\Gohan.jpg")

img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)

kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda x: x.distance)

img_matches = cv2.drawMatches(
    img1, kp1,
    img2, kp2,
    matches[:20],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

# -------------------- Image Alignment (Homography) --------------------
src_pts = np.float32([[50, 60], [100, 40], [80, 120], [150, 100]]).reshape(-1, 1, 2)
dst_pts = np.float32([[55, 65], [110, 50], [90, 130], [160, 110]]).reshape(-1, 1, 2)

M, _ = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
```

```
h, w = img1.shape[:2]
img_aligned = cv2.warpPerspective(img1, M, (w, h))

# -------------------- Display Results --------------------
plt.figure(figsize=(15, 12))

plt.subplot(4,2,1)
plt.title("Original Image")
plt.imshow(img)
plt.axis('off')

plt.subplot(4, 2, 2)
plt.title("Cloned Image")
plt.imshow(clon, cmap='gray')
plt.axis('off')

plt.subplot(4, 2, 3)
plt.title("FFT Magnitude Spectrum")
plt.imshow(magnitude_spectrum, cmap='gray')
plt.axis('off')

plt.subplot(4, 2, 4)
plt.title("Hough Lines")
plt.imshow(line_img)
plt.axis('off')
```

```
plt.subplot(4, 2, 5)
plt.title("Hough Circles")
plt.imshow(circle_img)
plt.axis('off')

plt.subplot(4, 2, 6)
plt.title("ORB Keypoints")
plt.imshow(img_keypoints)
plt.axis('off')

plt.subplot(4, 2, 7)
plt.title("Feature Matching")
plt.imshow(img_matches)
plt.axis('off')

plt.subplot(4, 2, 8)
plt.title("Image Alignment")
plt.imshow(img_aligned)
plt.axis('off')

plt.tight_layout()
plt.show()
```

EXP 5:

Image segmentation using graphcut / Grabcut

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
def graphcut_segmentation():
    image = cv2.imread(r"G:\CV_008\Jacki.jpg")
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    mask = np.zeros(image.shape[:2], np.uint8)
    bgdModel = np.zeros((1, 65), np.float64)
    fgdModel = np.zeros((1, 65), np.float64)
    h, w = image.shape[:2]
    rect = (10, 10, w-20, h-20)
    cv2.grabCut(image, mask, rect, bgdModel, fgdModel, 5,
cv2.GC_INIT_WITH_RECT)
    final_mask = np.where((mask == 2) | (mask == 0), 0, 1).astype("uint8")
    segmented_image = image * final_mask[:, :, np.newaxis]
    plt.figure(figsize=(10,5))
    plt.subplot(131)
    plt.imshow(image)
    plt.title("Original Image")
    plt.axis("off")
    plt.subplot(132)
    plt.imshow(final_mask, cmap='gray')
    plt.title("Mask")
```

```
    plt.axis("off")
    plt.subplot(133)
    plt.imshow(segmented_image)
    plt.title("Segmented Image")
    plt.axis("off")
    plt.tight_layout()
    plt.show()
if __name__ == "__main__":
    graphcut_segmentation()
```

EXP 6:

Camera calibration with circular grid

```
import numpy as np
import cv2 as cv
criteria=(cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER,30,0.001)
objp=np.zeros((6*7,3),np.float32)
objp[:,:2]=np.mgrid[0:7,0:6].T.reshape(-1,2)
objppoints=[]
imgpoints=[]
imgpath=r'G:\CV_008\Chessboard.jpg'
img=cv.imread(imgpath)
img=cv.resize(img,None,fx=2,fy=2)
cv.imshow('input image',img)
gray=cv.cvtColor(img,cv.COLOR_BGR2GRAY)
ret,corner=cv.findChessboardCorners(gray,(7,7),None)
if ret==True:
   objppoints.append(objp)
   corner2=cv.cornerSubPix(gray,corner,(11,11),(-1,1),criteria)
   imgpoints.append(corner2)
   cv.drawChessboardCorners(img,(7,7),corner2,ret)
cv.imshow('calibrated image',img)
cv.waitKey(0)
cv.destroyAllWindows()
```

EXP 7:

Face estimation

```
import cv2 as cv

import argparse


parser = argparse.ArgumentParser()

parser.add_argument('--input', help='Path to image or video. Skip to capture frames from camera')

parser.add_argument('--thr', default=0.2, type=float, help='Threshold value for pose parts heat map')

parser.add_argument('--width', default=368, type=int, help='Resize input to specific width.')

parser.add_argument('--height', default=368, type=int, help='Resize input to specific height.')


args = parser.parse_args()


BODY_PARTS = { "Nose": 0, "Neck": 1, "RShoulder": 2, "RElbow": 3, "RWrist": 4,

        "LShoulder": 5, "LElbow": 6, "LWrist": 7, "RHip": 8, "RKnee": 9,

        "RAnkle": 10, "LHip": 11, "LKnee": 12, "LAnkle": 13, "REye": 14,

        "LEye": 15, "REar": 16, "LEar": 17, "Background": 18 }


POSE_PAIRS = [ ["Neck", "RShoulder"], ["Neck", "LShoulder"], ["RShoulder", "RElbow"],

        ["RElbow", "RWrist"], ["LShoulder", "LElbow"], ["LElbow", "LWrist"],

        ["Neck", "RHip"], ["RHip", "RKnee"], ["RKnee", "RAnkle"], ["Neck", "LHip"],
```

```
        ["LHip", "LKnee"], ["LKnee", "LAnkle"], ["Neck", "Nose"], ["Nose", "REye"],
        ["REye", "REar"], ["Nose", "LEye"], ["LEye", "LEar"] ]

inWidth = args.width
inHeight = args.height

net = cv.dnn.readNetFromTensorflow(r'G:\CV_008\graph_opt1.pb')

cap = cv.VideoCapture(r'G:\CV_008\Sr.jpg')

while cv.waitKey(1) < 0:
  hasFrame, frame = cap.read()
  if not hasFrame:
    cv.waitKey()
    break

  frameWidth = frame.shape[1]
  frameHeight = frame.shape[0]

  net.setInput(cv.dnn.blobFromImage(frame, 1.0, (inWidth, inHeight), (127.5, 127.5, 127.5), swapRB=True, crop=False))
  out = net.forward()
  out = out[:, :19, :, :]  # MobileNet output [1, 57, -1, -1], we only need the first 19 elements

  assert(len(BODY_PARTS) == out.shape[1])
```

```
points = []
for i in range(len(BODY_PARTS)):
    # Slice heatmap of corresponging body's part.
    heatMap = out[0, i, :, :]

    # Originally, we try to find all the local maximums. To simplify a sample
    # we just find a global one. However only a single pose at the same time
    # could be detected this way.
    _, conf, _, point = cv.minMaxLoc(heatMap)
    x = (frameWidth * point[0]) / out.shape[3]
    y = (frameHeight * point[1]) / out.shape[2]
    # Add a point if it's confidence is higher than threshold.
    points.append((int(x), int(y)) if conf > args.thr else None)

for pair in POSE_PAIRS:
    partFrom = pair[0]
    partTo = pair[1]
    assert(partFrom in BODY_PARTS)
    assert(partTo in BODY_PARTS)

    idFrom = BODY_PARTS[partFrom]
    idTo = BODY_PARTS[partTo]

    if points[idFrom] and points[idTo]:
        cv.line(frame, points[idFrom], points[idTo], (0, 255, 0), 3)
```

```
        cv.ellipse(frame, points[idFrom], (3, 3), 0, 0, 360, (0, 0, 255), cv.FILLED)
        cv.ellipse(frame, points[idTo], (3, 3), 0, 0, 360, (0, 0, 255), cv.FILLED)

    t, _ = net.getPerfProfile()
    freq = cv.getTickFrequency() / 1000
    cv.putText(frame, '%.2fms' % (t / freq), (10, 20),
cv.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0))

    cv.imshow('OpenPose using OpenCV', frame)
    key = cv.waitKey(1)
    if key == ord('q'):
        break

# release the file pointers
cv.destroyAllWindows()
cap.release()
```

EXP 8:

3D Reconstruction – creating Depth map from Stereo images

```
import cv2
import numpy as np
from matplotlib import pyplot as plt


# -------------------------------
# 1. LOAD AS GRAYSCALE (IMPORTANT)
# -------------------------------
imgL = cv2.imread("im0.png", cv2.IMREAD_GRAYSCALE)
imgR = cv2.imread("im1.png", cv2.IMREAD_GRAYSCALE)


if imgL is None or imgR is None:
    raise Exception("Error loading images")


# -------------------------------
# 2. FIX SIZE (MANDATORY)
# -------------------------------
h = min(imgL.shape[0], imgR.shape[0])
w = min(imgL.shape[1], imgR.shape[1])
```

```
imgL = imgL[:h, :w]
imgR = imgR[:h, :w]

# -------------------------------
# 3. ENSURE TYPE uint8
# -------------------------------
imgL = np.uint8(imgL)
imgR = np.uint8(imgR)

# -------------------------------
# 4. STEREO MATCHING
# -------------------------------
stereo = cv2.StereoSGBM_create(
    minDisparity=0,
    numDisparities=16 * 6,
    blockSize=5
)

disparity = stereo.compute(imgL, imgR).astype(np.float32) / 16.0

# -------------------------------
# 5. DEPTH MAP
# -------------------------------
disparity[disparity < 0] = 0
```

```
disp = cv2.normalize(disparity, None, 0, 255, cv2.NORM_MINMAX)
disp = np.uint8(disp)


# ------------------------------
# 6. DISPLAY
# ------------------------------
plt.imshow(disp, cmap='gray')
plt.title("Depth Map")
plt.axis('off')
plt.show()


# ------------------------------
# 7. SAVE
# ------------------------------
cv2.imwrite("depth_map.png", disp)


print("Saved: depth_map.png")
```

EXP 9:

Object Detection and Tracking

Kalman Filter

```
import cv2
import numpy as np

cap = cv2.VideoCapture("Traffic IP Camera video.mp4")

# Background subtractor
fgbg = cv2.createBackgroundSubtractorMOG2(
    history=300,
    varThreshold=25,
    detectShadows=True
)

# Tracker dictionary
trackers = {}
next_id = 0

# Parameters
distance_threshold = 80
min_area = 1500
max_missed = 5
```

```
# Kalman filter
def create_kalman(x, y):
    kf = cv2.KalmanFilter(4, 2)
    kf.measurementMatrix = np.array([[1,0,0,0],[0,1,0,0]], np.float32)
    kf.transitionMatrix = np.array([[1,0,1,0],
                                    [0,1,0,1],
                                    [0,0,1,0],
                                    [0,0,0,1]], np.float32)
    kf.processNoiseCov = np.eye(4, dtype=np.float32) * 0.03
    kf.statePre = np.array([[x],[y],[0],[0]], np.float32)
    return kf

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.resize(frame, (960, 540))

    # Reset predictions each frame
    for data in trackers.values():
        data.pop('pred', None)

    fgmask = fgbg.apply(frame)

    # Remove shadows
```

```
_, fgmask = cv2.threshold(fgmask, 200, 255, cv2.THRESH_BINARY)

# Morphology
fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, np.ones((3,3),
np.uint8))
fgmask = cv2.dilate(fgmask, np.ones((5,5), np.uint8), iterations=2)

# Contour detection
contours, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

detections = []
for cnt in contours:
    area = cv2.contourArea(cnt)
    if area > min_area:
        x, y, w, h = cv2.boundingRect(cnt)
        cx, cy = x + w//2, y + h//2
        detections.append((cx, cy, x, y, w, h))

updated_trackers = {}
matched_ids = set()

# Matching
for det in detections:
    cx, cy, x, y, w, h = det
```

```
min_dist = float('inf')
matched_id = None

for tid, data in trackers.items():
    if tid in matched_ids:
        continue

    # Predict once
    if 'pred' not in data:
        data['pred'] = data['kf'].predict()

    pred = data['pred']
    px, py = int(pred[0].item()), int(pred[1].item())

    dist = np.sqrt((cx - px)**2 + (cy - py)**2)

    if dist < min_dist and dist < distance_threshold:
        min_dist = dist
        matched_id = tid

if matched_id is not None:
    kf = trackers[matched_id]['kf']
    kf.correct(np.array([[np.float32(cx)], [np.float32(cy)]]))

    # Smooth bounding box
    old_box = trackers[matched_id]['bbox']
```

```
    alpha = 0.7
    new_box = [x, y, w, h]

    smoothed_box = [
        int(alpha*old_box[0] + (1-alpha)*new_box[0]),
        int(alpha*old_box[1] + (1-alpha)*new_box[1]),
        int(alpha*old_box[2] + (1-alpha)*new_box[2]),
        int(alpha*old_box[3] + (1-alpha)*new_box[3]),
    ]

    updated_trackers[matched_id] = {
        'kf': kf,
        'bbox': smoothed_box,
        'missed': 0
    }

    matched_ids.add(matched_id)

else:
    kf = create_kalman(cx, cy)
    updated_trackers[next_id] = {
        'kf': kf,
        'bbox': [x, y, w, h],
        'missed': 0
    }
    next_id += 1
```

```
# Handle unmatched trackers
for tid, data in trackers.items():
    if tid not in matched_ids:
        data['missed'] += 1
        if data['missed'] <= max_missed:
            updated_trackers[tid] = data

trackers = updated_trackers

# Draw results
for tid, data in trackers.items():
    x, y, w, h = data['bbox']

    if 'pred' not in data:
        data['pred'] = data['kf'].predict()

    pred = data['pred']
    px, py = int(pred[0].item()), int(pred[1].item())

    cv2.rectangle(frame, (x, y), (x+w, y+h), (0,255,0), 2)
    cv2.circle(frame, (px, py), 5, (0,0,255), -1)
    cv2.putText(frame, f"ID {tid}", (x, y-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,255,0), 2)

cv2.imshow("Car Tracking", frame)
```

```
    if cv2.waitKey(30) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Camshaft Tracking

```
import cv2

# Open video
cap = cv2.VideoCapture("videoplayback.mp4")

# Read first frame
ret, frame = cap.read()
if not ret:
    print("❌ Cannot read video")
    exit()

frame = cv2.resize(frame, (960, 540))

# Select ROI
bbox = cv2.selectROI("Select Object to Track", frame, False, False)
x, y, w, h = bbox
```

```
if w == 0 or h == 0:
    print("❌ No ROI selected")
    exit()

# Create CSRT tracker (VERY IMPORTANT)
tracker = cv2.TrackerCSRT_create()

# Initialize tracker with first frame + bbox
tracker.init(frame, bbox)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.resize(frame, (960, 540))

    # Update tracker
    success, bbox = tracker.update(frame)

    if success:
        x, y, w, h = [int(v) for v in bbox]

        # Draw rectangle
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

```
    # Center point
    cx = x + w // 2
    cy = y + h // 2
    cv2.circle(frame, (cx, cy), 5, (0, 0, 255), -1)

    cv2.putText(frame, "Tracking", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
  else:
    cv2.putText(frame, "Lost Object", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)

  cv2.imshow("Object Tracker (CSRT)", frame)

  if cv2.waitKey(30) & 0xFF == 27:
    break

cap.release()
cv2.destroyAllWindows()
```

EXP 10:

Face detection

```
import cv2
img = cv2.imread("grp.jpg")
if img is None:
    print("Image not found")
    exit()
# Resize for better detection
img = cv2.resize(img, None, fx=1.5, fy=1.5)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Improve contrast
gray = cv2.equalizeHist(gray)


face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=3,
    minSize=(30, 30)
)
```

```
print("Faces detected:", len(faces))
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.imshow("Faces", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```